



# 01 Bottom line up front

## RECOMMENDATION

**Give each product its own thin, readable workflow "spine"** that calls **shared** domain steps. Make product differences visible in the model, not buried in config and code. Migrate the legacy monoliths onto it, highest-volume first.

## THE REFRAME

The premise was "one giant workflow serves every product." In production that is **false**. The shared workflow runs **one** product at **5.5%** of volume. The giants are the **legacy per-product monoliths** carrying **94%**.

## THE DECISION FOR YOU

Not "unwind a giant shared workflow." It is: the shared design has proven safe on DF4W. **Do we migrate the legacy book onto it** — with product-owned spines and explicit variation rather than today's hidden config?

## 02 What actually runs in production

Direct queries against the production BPM database, 90 days to Sep 2026. Not estimates.

94%

of new loans run on **legacy per-product monoliths**

5.5%

run on the shared "unified" spine

1

product (DF4W) is actually on the shared spine

~250<sub>k</sub>

NDF2W loans per quarter, on one legacy monolith

**The sharp finding.** "A single workflow for all products" is, in production, the **DF4W workflow**. NDF2W, NDF4W, RO and Sharia — the entire live retail book — still run the giant per-product processes. The shared spine carries one product plus a 3-application pilot.

### 03 Which workflows are actually "giant"?

Size and churn are in the legacy per-product processes — the exact design the premise proposed — not in the shared spine.

| WORKFLOW                      | DESIGN                  | DIAGRAM NODES | CODE CHANGES | PRODUCTS |
|-------------------------------|-------------------------|---------------|--------------|----------|
| <b>NDF4W</b> monolith         | legacy, per product     | 223           | 392          | 1        |
| <b>NDF2W</b> monolith         | legacy, per product     | 174           | 359          | 1        |
| <b>Shared "unified" spine</b> | one graph, all products | 43            | 36           | 1 live   |

● The two legacy monoliths hold **145** distinct step implementations for **2** products.

● The shared spine and its children hold **69** for **5** configured products. The shared design is smaller, not bigger.

04

# The real gap: product logic is invisible

On the shared spine, what a product actually does is assembled from **four** places at once. None of them is the diagram.

## Database config

A row can silently switch a step to a **no-op**. **55** steps are gated this way.

## Java branches

`if (application.isDF4W())`  
inside **19 of 68** shared steps;  
`isDF4W()` alone appears 20×.

## BPMN gateways

3 product conditions embedded in the process XML.

## Feature flags

9 flags in app config decide half-on / half-off per environment.

**Measured, not hypothetical.** On the shared spine, checks that don't apply to a product still appear in history as completed tasks — finishing in **~43 ms** (a no-op) versus **900–1,900 ms** for a real credit-bureau call. The diagram shows steps that never ran.

# Why this costs you

## Blast radius

One shared graph. A DF4W change redeploys the process every product runs. Changes to shared survey logic have touched the flow NDF2W depends on.

## No owner-readable artifact

A product owner cannot be handed a diagram of *their* product. The differences live in SQL and Java, not a picture.

## Hidden test matrix

Correctness of one file depends on products × config rows × flags. Repeated "set-active / set-inactive" fix-up migrations are that matrix leaking.

## Silent divergence

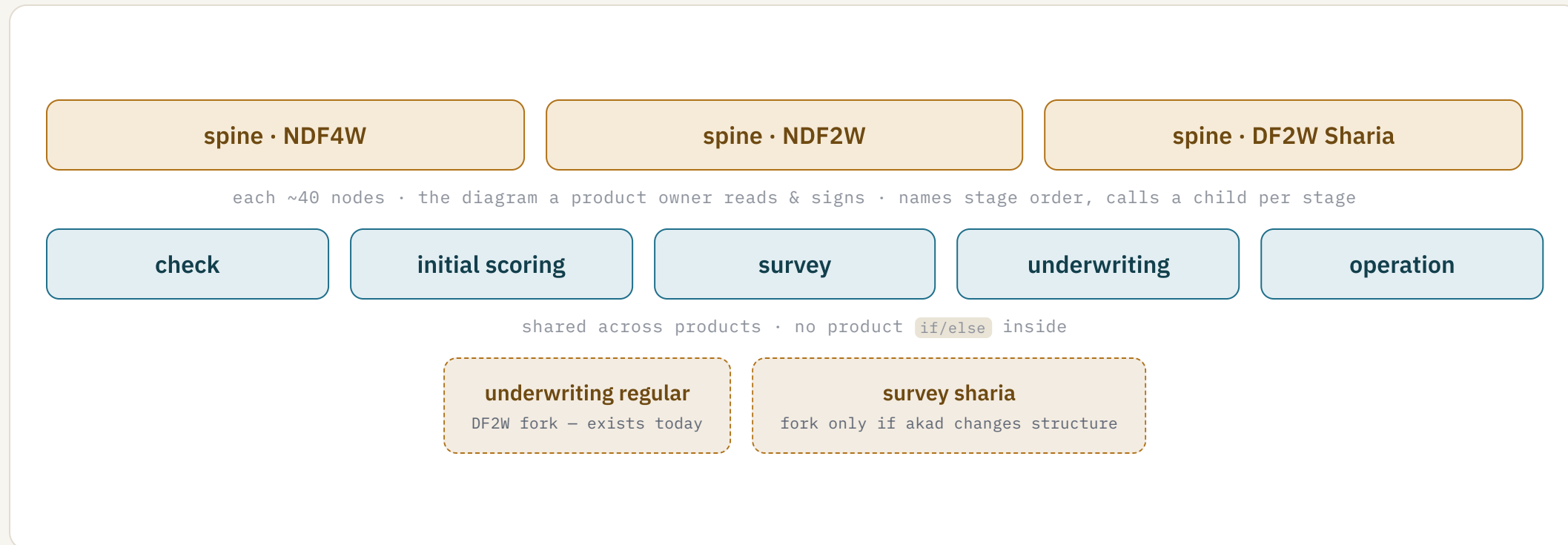
A missing config row doesn't fail a deploy. It quietly removes a credit check from a loan product. No alarm.

### WORTH KEEPING

Human work is already isolated in leaf steps; domain steps largely exist once; escalation and cancellation are centralised. The foundations are right.

## 06 Best practice: the target shape

Product-owned spines. Domain-owned shared children. Fork only where the *structure* genuinely differs. Standard orchestrator / worker separation.



Camunda already supports this cheaply: a step can pick its child by expression, so one spine calls `survey` and another `survey_sharia` with no gateway and no flag.

# Gap to best practice, by severity

| PRINCIPLE                        | BRAVO TODAY                                                     | SEVERITY |
|----------------------------------|-----------------------------------------------------------------|----------|
| Model is the source of truth     | Structure hidden as config no-ops; needs a 5-table join to read | High     |
| Thin owned spine per product     | One shared graph; product chosen by gateways + a variable       | High     |
| No product logic in shared steps | 19 of 68 shared steps branch on product                         | High     |
| Product out of domain code       | Product lives in config + Java + gateways + flags at once       | High     |
| Least-privilege engine           | Workflow console was open; 61 hostile definitions were injected | Security |
| Migrate by strangler             | Happening by accident; legacy still carries 94%                 | Medium   |

**In one sentence:** the building blocks are right, but product variation hides in three places the model can't show, so no product has a readable, owned, deployable spine — and the monoliths carrying 94% of volume were never migrated.



# The plan: 30 / 60 / 90 days

0-30d

## See it & stop the bleeding

- Publish an auto-generated **flow diagram per product** from the config
- CI guard: block any **new** hidden product branch or no-op gate
- Harden the workflow engine console (**security prerequisite**)
- Sign off the target shape; delete dead config

**Exit:** every live product has an owner-readable flow

30-60d

## Prove it on DF4W

- Build an explicit **spine · DF4W** (already the only live spine product = lowest risk)
- Remove its product gateways and hidden branches
- Turn config no-ops into real, visible model steps
- Per-product diagram diffed on every build

**Exit:** DF4W runs a thin owned spine, zero phantom tasks

60-90d

## Migrate the big one

- Stand up **spine · NDF2W** (≈250k loans/quarter) on the shared children
- Strangler-route rising % of new NDF2W volume
- De-duplicate the worst 2W/4W step pairs
- Write the decommission plan for NDF4W / RO / Sharia

**Exit:** ≥25% of new NDF2W on the spine at parity

## 09 Guardrails, scope, and one prerequisite

### PREREQUISITE — DO FIRST

**Harden the Camunda console.** The workflow engine's admin surface was open to the network; **61 malicious process definitions were injected into production and one executed.** This is written up separately. It must be closed before, or alongside, day-one work — the migration adds deployment activity to the same surface.

### WHY THIS SEQUENCE IS LOW-RISK

We prove the pattern on DF4W — the product already running the shared spine — before touching the high-volume legacy book. Nothing is big-bang; legacy stays live for rollback until its replacement carries real traffic cleanly.

### OUT OF SCOPE FOR 90 DAYS

Rewriting survey/underwriting *business logic*; the separate Sharia deployment; and the LORA/Temporal track. This programme changes only the **shape** of the Bravo workflows — moving product logic out of hiding onto owned, readable spines.

# The ask

Approve the 30/60/90 programme and the target architecture: **product-owned spines, shared domain children, explicit variation.**

Phase 1 is visibility and safety only — no workflow moves. The migration decision (Phase 3) comes back to you with DF4W proven and real metrics in hand.

## WE REPORT MONTHLY

% of volume on explicit product spines

0 → all live products

Hidden product branches in shared code

19 → 0

Phantom no-op tasks per day

heavy → ~0

Duplicated step implementations

145 → falling